UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

PRODUCTO FINAL UNIDAD I

Presentado por:

- **Checalla Alata, Cristian Alex**
- **Condori Tarqui William Martin**
- **Huacasi Sejje, Anthony Jose**

Curso: Programación Orientada a Objetos I

Ingeniero: Miguel Romilio, Aceituno Rojo

Semestre: Segundo

Grupo: “A”

Puno – Perú – 2026

ÍNDICE

INTRODUCCIÓN **3**

1. Objetivo general **3**

2. Explicación del código **3**

2.1. Herramientas utilizadas 3

2.2. Organización general 4

2.3. Datos precargados de la tienda 5

2.4. Controles de la aplicación 5

2.5. Abstracción 6

2.6. Herencia 6

2.7. Polimorfismo 7

2.8. Encapsulamiento 7

2.9. Composición y diseño abierto a extensión 8

2.10. Cálculo del total de una compra 8

2.11. Persistencia del inventario 8

2.12. Interfaz gráfica 8

3. Instalación y ejecución del programa **9**

3.1. Requisitos y estructura de carpetas 9

3.2. Ejecutar con Code::Blocks (Windows) 10

3.2.1. Instalar MSYS2 10

3.2.2. Instalar las dependencias 10

3.2.3. Crear el proyecto 11

3.2.4. Configurar el compilador 11

3.2.5. Enlazar las librerías 12

3.2.6. Compilar y ejecutar 13

4. Ejemplo de funcionamiento **14**

4.1. Paso a paso en la aplicación 14

4.2. Cálculo del total 16
4.3. Cupones disponibles 17

5. Funciones del código **18**
5.1. Utilidades básicas 18
5.2. Dominio: personas 19
5.3. Dominio: productos 19
5.3.1. Funciones de apoyo para los modelos 19
5.3.2. Clase Producto (base) 20
5.3.3. Clases hijas de producto 21
5.4. Dominio: pago, comprobante, envío, promociones y cupones 22
5.4.1. Clase Carrito 23
5.5. Dominio: clase Tienda 23
5.6. Interfaz: funciones de dibujo 25
5.7. Interfaz: elementos reutilizables 26
5.8. Clase Interfaz 27
5.8.1. Utilidades 27
5.8.2. Páginas y ventanas 28
5.8.3. Ciclo principal 29

6. Limitaciones y mejoras futuras **29**
6.1. Funcionalidad 29
6.2. Estructura del código 30
6.3. Interfaz 30

CONCLUSIONES **30**

ANEXOS **31**

INTRODUCCIÓN

El presente informe presenta los logros aprendidos durante el trayecto de la Primera Unidad en el curso de Programación Orientada a Objetos (POO), en donde se desarrolló un menú interactivo de una tienda virtual con diversas funciones integradas, el nombre de la tienda es TecnoStore, una aplicación de escritorio que simula una tienda de computadoras, en donde el cliente puede navegar un catálogo de productos, elegir el modelo que desea, armar un carrito con promociones automáticas, aplicar cupones, pagar con distintos medios y obtener un comprobante. Donde también el programa permite la introducción de nuevos productos o aumentar el inventario existente.

En el presente informe se expondrá a detalle en cómo se desarrolló, con que medios se apoyó, en cómo funciona dicho código, entre otros datos a mencionar, junto a como acceder dicho código.

1. Objetivo general

Desarrollar una tienda virtual, desde el lenguaje de programación C++ usando una interfaz gráfica que permita simular la venta de una tienda de computadoras, aplicando los cuatro pilares de la Programación Orientada a Objetos: abstracción, herencia, polimorfismo y encapsulamiento.

2. Explicación del código

2.1. Herramientas utilizadas

Antes de explicar cómo funciona el código se describen los recursos empleados:

- **Lenguaje:** C++ con el estándar C++17 (`-std=c++17`).
- **Librería gráfica:** raylib, que permite dibujar ventanas, menús, animaciones y capturar el mouse y el teclado de forma sencilla.
- **Compilador y entorno:** GCC (MinGW-w64) instalado mediante MSYS2.
- **IDE:** se usaron dos entornos, Code::Blocks y Visual Studio Code, para la estructura y depuración del código.
- **Diseño:** XMind para elaborar el diagrama de clases del proyecto a partir de los fundamentos de POO.
- **Control de versiones:** GitHub para alojar el repositorio del proyecto.

Todo el código funciona en base a los cuatro pilares de la POO: encapsulamiento, abstracción, herencia y polimorfismo.

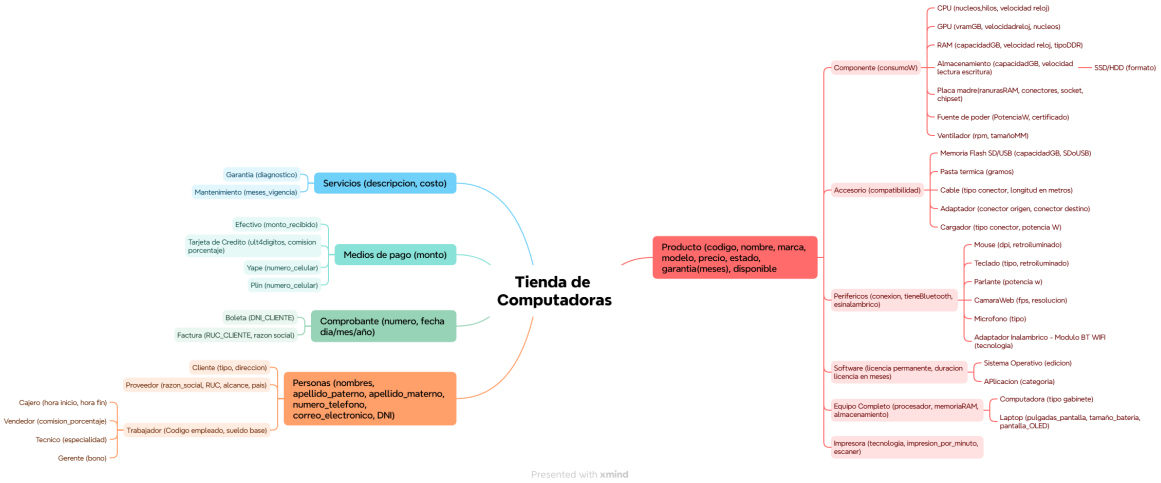
2.2. Organización general

El programa está en un solo archivo (`main.cpp`) de aproximadamente 4 250 líneas, dividido en dos capas: el **dominio** (las clases que modelan el negocio y no dependen de la pantalla) y la **interfaz** (todo lo que usa raylib para dibujar). El archivo sigue siete secciones, en este orden:

N.°	Sección	Contenido
1	Utilidades	Funciones auxiliares: formato de dinero, mayúsculas y minúsculas, separación de textos.
2	Personas	`Persona` y sus hijas: `Cliente`, `Proveedor` y `Trabajador` (`Cajero`, `Vendedor`, `Tecnico`, `Gerente`).
3	Productos	`Producto` y sus familias: componentes, accesorios, periféricos, software, equipos completos y servicios.
4	Pago, comprobante, envío, promociones, cupones y carrito	Reglas de la venta, cada una como clase abstracta con sus hijas.
5	Tienda	Clase que reúne todo el dominio, calcula totales, cierra la venta y guarda el inventario.
6	Interfaz (raylib)	Funciones de dibujo, ilustraciones de productos, pantallas y la clase `Interfaz`.
7	`main()`	Crea la `Interfaz` y la ejecuta.

La estructura de todo el código se resume en la siguiente imagen:

Figura 1
Diagrama de la tienda de computadoras



Nota. Elaboración propia (desarrollado desde XMind).

2.3. Datos precargados de la tienda

Al iniciar, la clase `Tienda` crea automáticamente los datos con los que se puede probar el sistema:

Elemento	Detalle
Catálogo base	39 productos en 6 categorías: Hardware (15), Accesorios (7), Periféricos (7), Software (4), Equipos (3) y Servicios (3).
Modelos por producto	Cada producto tiene varios modelos con un factor que multiplica el precio base (por ejemplo, un teclado "Mecánico" vale 1,4 veces el precio base); uno de ellos se marca como recomendado.
Medios de pago	Efectivo, Tarjeta de crédito (recargo de 3 %), Yape y Plin.
Comprobantes	Boleta (serie B001) y Factura (serie F001, con RUC e IGV).
Envíos	Recojo en tienda (gratis) y Delivery (S/ 10).
Promociones	Combo Mouse inalámbrico + Teclado (S/ 10 por pareja) y descuento por volumen (5 % desde 3 unidades).
Cupones	Cinco códigos de descuento (ver la sección de ejemplo).
Personal	Una gerente, dos vendedores, un cajero y un técnico.
Proveedores	Uno nacional (Perú) y uno internacional (China).

La aplicación se organiza en siete pestañas: **Inicio**, **Productos**, **Ofertas**, **Servicios**, **Pedidos**, **Nosotros** e **Inventario**. La pestaña Ofertas muestra los productos marcados como oferta (el Mouse inalámbrico, el Teclado y todo el software), y la pestaña Servicios muestra solo la categoría del mismo nombre.

2.4. Controles de la aplicación

- **F11:** activa o desactiva la pantalla completa.
- **Esc:** cierra la ventana emergente abierta; si no hay ninguna, limpia el buscador.
- **Tab:** pasa al siguiente campo durante el proceso de compra.
- **Enter:** en la ventana de compra aplica el cupón (si el cursor está en ese campo) o avanza al siguiente paso.
- **Rueda del mouse:** desplaza las páginas y las listas largas de modelos.

2.5. Abstracción

Una clase abstracta define qué se debe hacer sin decir cómo. En el desarrollo de dicho proyecto en base a C++ se basa principalmente en:

- **Producto:** obliga a cada producto a implementar `detalle()`.
- **Persona:** obliga a implementar `rol()`.
- **MedioPago, Comprobante, Envio, Promocion, Cupon:** cada una define un conjunto de operaciones que las hijas completan.
- **ElementoUI:** base de los elementos dibujables de la interfaz.

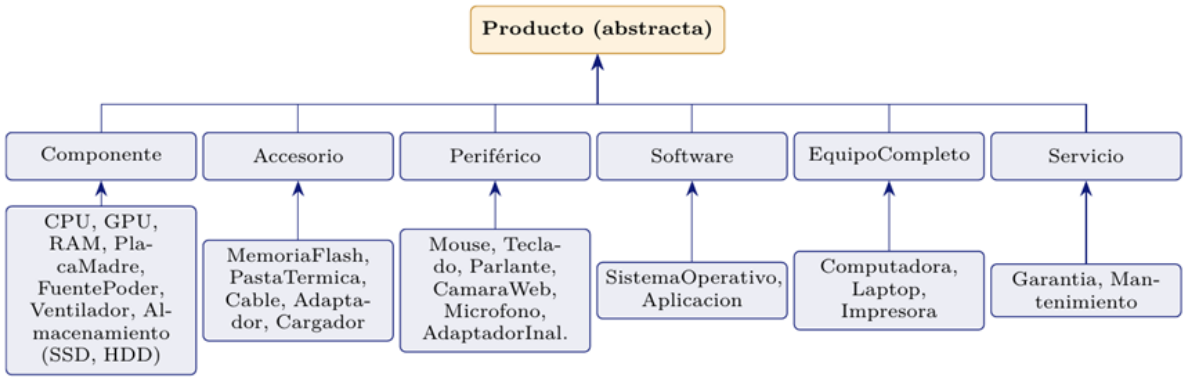
```
class Producto {
    ...
    virtual double precioFinal(int m = 0) const { return
   precioModelo(m); }
    virtual string detalle() const = 0;
};
```

2.6. Herencia

Las clases hijas reutilizan lo que ya tiene la clase padre y solo agregan lo propio.

En el contexto del proyecto la parte más importante entre clases padre e hijas del mismo es compuesta desde el producto, el cual es representado desde la siguiente imagen:

Figura 2
Diagrama de herencias y clases (Producto)



Nota. Elaboración propia.

De forma parecida entre otras aplicaciones de herencias pero en este caso de menor complejidad y/o extensión (clase padre a hija) son:

- **Personas:** Persona → Cliente, Proveedor, Trabajador → Cajero, Vendedor, Tecnico, Gerente.
- **Pagos:** MedioPago → Efectivo, Tarjeta, Billetera → Yape, Plin.
- **Cupones:** Cupon → CuponPorcentaje, CuponMonto.

2.7. Polimorfismo

Un mismo método se comporta distinto según el objeto real. El programa guarda punteros a la clase base y, al llamar el método, C++ ejecuta la versión de la clase hija.

Método	Comportamiento según la clase
`detalle()`	Cada producto describe sus propias características.
`precioFinal()`	Software aplica −5 %; EquipoCompleto suma S/ 10 por cada mes de garantía.
`recargo()`	Tarjeta suma 3 %; Efectivo, Yape y Plin no suman nada.
`validar()`	Efectivo revisa el monto; Tarjeta, 4 dígitos; billeteras, celular de 9 dígitos.
`comision()`	Vendedor y Gerente calculan su porcentaje; los demás devuelven 0.
`emitir()`	Factura incluye el IGV; Boleta no.

```
class Tarjeta : public MedioPago {
public:
    double recargo(double m) const override { return m * 0.03; } //
   +3%
};

double recargo() const { return medios_[medio_]->recargo(base()); }
```

2.8. Encapsulamiento

Los atributos son `protected` o `private` y se consultan con métodos (getters) como `stock()`, `precioBase()` o `codigo()`. Por ejemplo, el stock solo cambia mediante `quitarStock()` y `fijarStock()`, que impiden valores negativos.

```
void quitarStock(int n) { stock_ = max(0, stock_ - n); }
void fijarStock(int n)  { stock_ = max(0, n); }
```

2.9. Composición y diseño abierto a extensión

- **Composición (relación "tiene un"):** la clase `Tienda` contiene el catálogo, el carrito, los trabajadores, los proveedores, los medios de pago, comprobantes, envíos y cupones.
- **Principio abierto/cerrado:** para agregar un nuevo medio de pago o un nuevo tipo de cupón basta crear una clase hija y registrarla; el resto del programa no cambia.
- **Memoria segura:** los objetos se guardan en `vector<unique_ptr<...>`, de modo que se liberan solos, sin usar `delete`.

2.10. Cálculo del total de una compra

El total se obtiene en tres pasos dentro de `Tienda`:

- **Subtotal:** suma de precio del modelo $\times$ cantidad de cada línea del carrito.
- **Base:** subtotal $-$ promociones $-$ cupón $+$ costo de envío.
- **Total:** base $+$ recargo del medio de pago.

```
double base() const { return carrito_.subtotal() - carrito_.
   descuentos()
                              - descuentoCupon() + costoEnvio; }
double recargo() const { return medios_[medio_]->recargo(base()); }
double total() const { return base() + recargo(); }
```

2.11. Persistencia del inventario

Al vender o reponer, el programa escribe en `img/inventario.dat` una línea por producto:

- `S|codigo|stock` para productos del catálogo base (solo guarda el stock).
- `P|codigo|nombre|marca|...` para productos creados por el usuario (guarda todos sus datos).

Al iniciar, `cargarInventario()` lee el archivo y restaura el estado. Las filas dañadas se ignoran sin detener el programa. Los pedidos realizados y la numeración de los comprobantes **no** se guardan: viven en memoria mientras la aplicación está abierta.

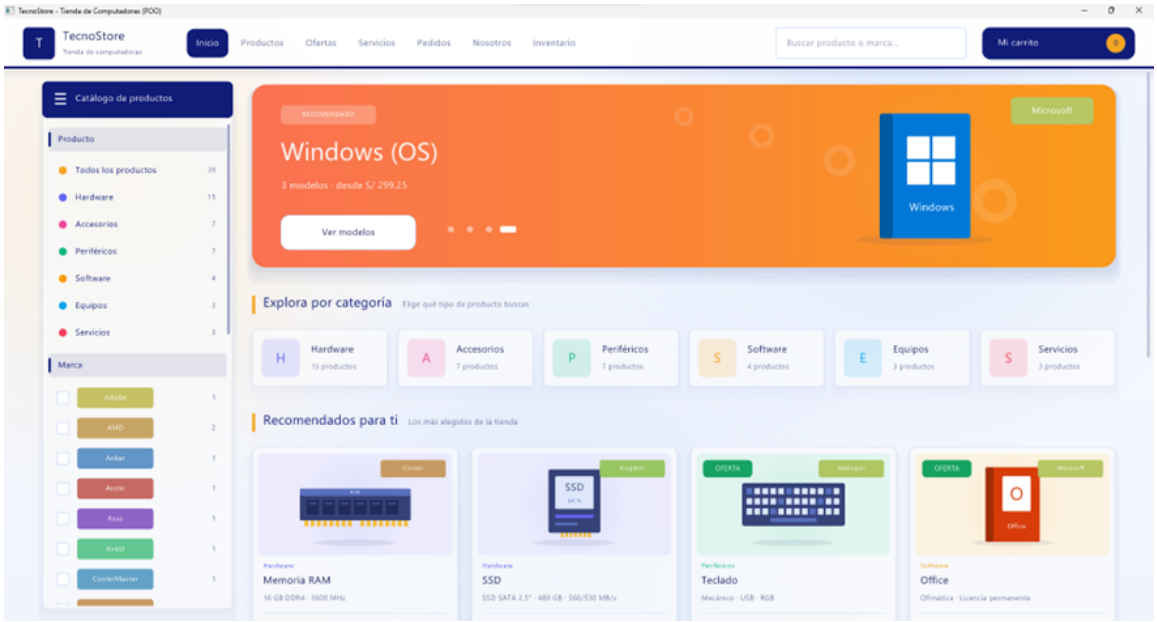
2.12. Interfaz gráfica

La interfaz funciona con un ciclo principal que se repite 60 veces por segundo (`paso()`). En cada cuadro:

- Lee el mouse, la rueda y el teclado, y ajusta el tamaño a la ventana.
- Anima los paneles y los efectos al pasar el mouse.
- Dibuja las capas en orden: fondo, página, barra superior, carrito, pago, detalle de producto y aviso.
- Ejecuta las acciones pendientes guardadas con `luego()`, para no modificar datos mientras se dibuja.

Las pantallas se organizan por capas: 0 página, 1 carrito, 2 pago y 3 detalle de producto. Solo la capa activa recibe los clics. Si un producto no tiene foto (`img/productos/codigo.png`), la función `arte()` dibuja una ilustración hecha con formas geométricas.

Figura 3
Menú de la aplicación



Nota. Captura de la pantalla principal de TecnoStore.

3. Instalación y ejecución del programa

Antes de mostrar el funcionamiento de la tienda, se explica cómo compilar y ejecutar el proyecto en Windows. Se eligió Code::Blocks como entorno de desarrollo, con el compilador GCC y la librería raylib instalados mediante MSYS2.

3.1. Requisitos y estructura de carpetas

- Sistema operativo Windows con conexión a Internet para la instalación inicial.
- MSYS2 en su ruta por defecto (`C:\msys64`), con el compilador GCC, `make` y raylib.

- Code::Blocks como IDE.
- El archivo `main.cpp` del proyecto, con el estándar C++17 activado.

Junto al programa ejecutable debe existir la carpeta `img/`, que el programa usa de la siguiente manera:

Ruta	Uso
`img/productos/<codigo>.png`	Foto opcional de cada producto (por ejemplo, `CPU-01.png`). Si no existe, el programa dibuja una ilustración con formas geométricas.
`img/marcas/<marca>.png`	Logo opcional de cada marca. Si no existe, se muestra una etiqueta de color con el nombre.
`img/inventario.dat`	Archivo donde se guarda el stock y los productos creados por el usuario; se genera solo al vender o reponer.

Importante. La carpeta `img/` debe existir en el directorio desde donde se ejecuta el programa. Si no existe, el inventario no se puede guardar y los cambios de stock se pierden al cerrar la aplicación.

3.2. Ejecutar con Code::Blocks (Windows)

Si se prefiere trabajar con un IDE, se deben seguir los pasos siguientes en el orden indicado. Todos son necesarios, en especial los de MSYS2.

3.2.1. Instalar MSYS2

Descargar MSYS2 desde `https://www.msys2.org` e instalarlo en el directorio `C:\msys64`, que es la ruta por defecto.

3.2.2. Instalar las dependencias

Abrir **MSYS2 UCRT64** desde el menú Inicio y ejecutar los siguientes comandos:

```
pacman -Syu
pacman -S mingw-w64-ucrt-x86_64-gcc
pacman -S mingw-w64-ucrt-x86_64-raylib
pacman -S mingw-w64-ucrt-x86_64-make
```

Nota. Si la terminal se cierra al terminar `pacman -Syu`, se debe abrir de nuevo y continuar con los demás comandos.

3.2.3. Crear el proyecto

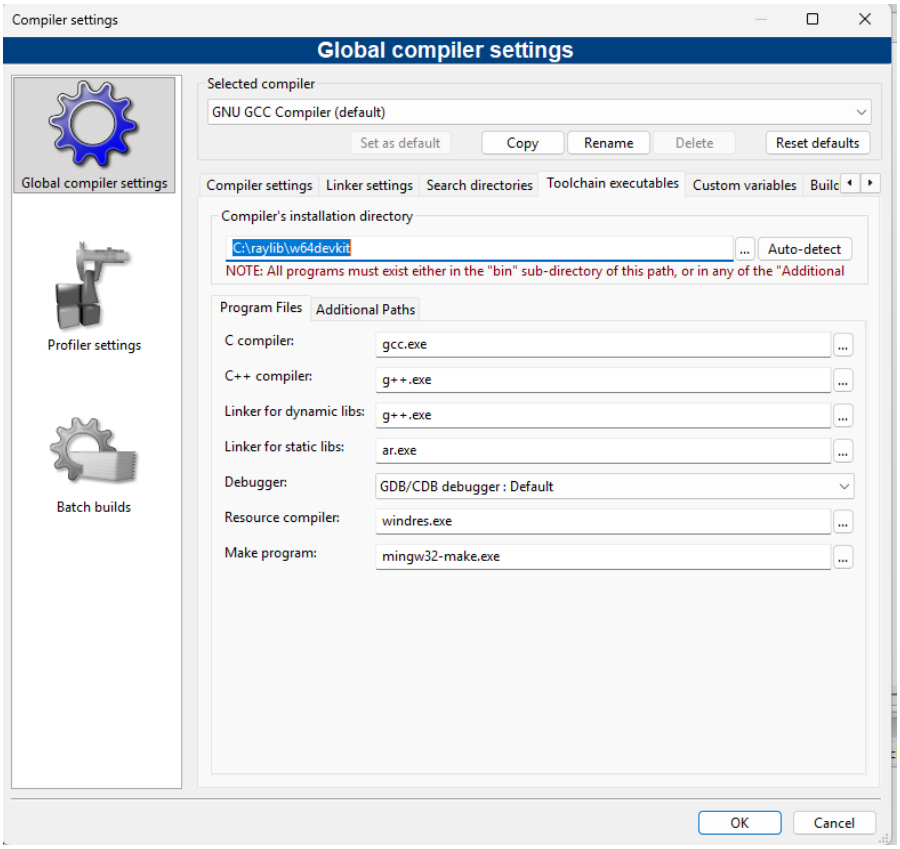
1. En Code::Blocks, ir a **File** → **New** → **Project** y elegir **Console application**.
2. Seleccionar el lenguaje **C++**.
3. Eliminar el `main.cpp` que se crea por defecto y agregar el archivo `main.cpp` del proyecto: clic derecho sobre el proyecto → **Add files...**

3.2.4. Configurar el compilador

Ir a **Settings** → **Compiler** y configurar lo siguiente:

1. En **Selected compiler**, dejar el predeterminado: **GNU GCC Compiler**.
2. En **Compiler settings** → **Compiler Flags**, marcar la opción **Have g++ follow the C++17 ISO C++ language standard [-std=c++17]**.
3. En la pestaña **Toolchain executables**, escribir en **Compiler's installation directory**: `C:\msys64\ucrt64`.

Figura 4
Configuración de Code::Blocks



Nota. Captura de la configuración del compilador (opcional).

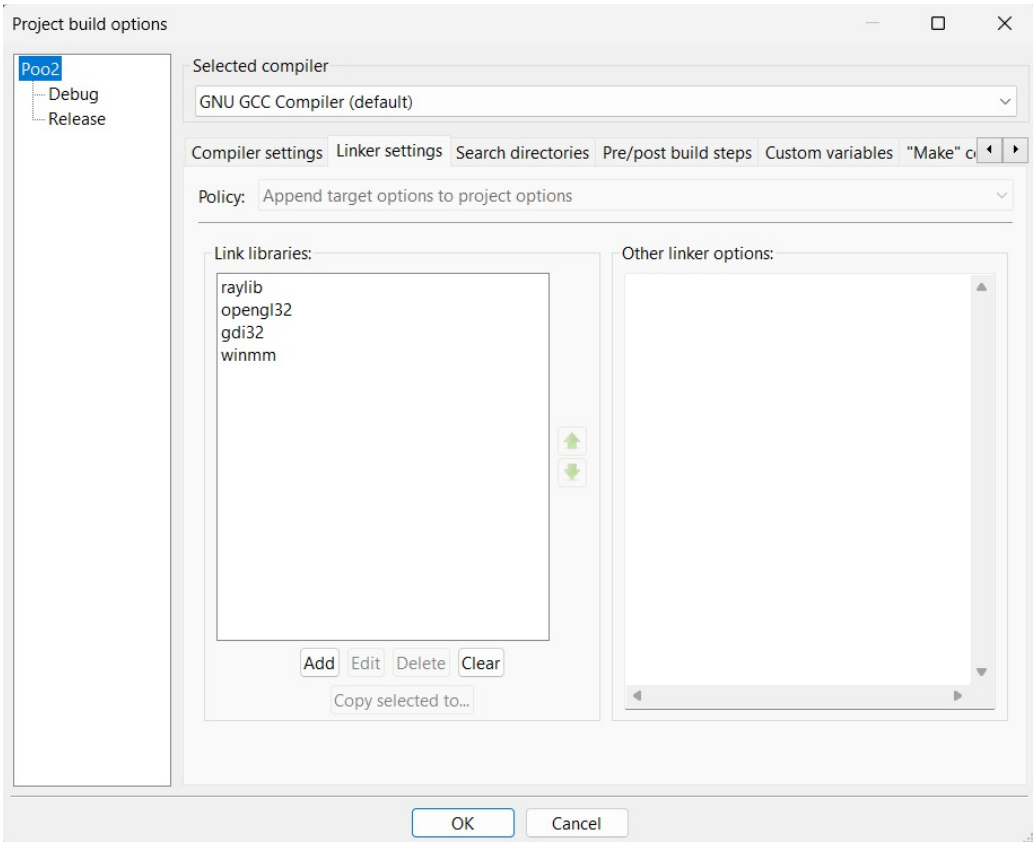
4. Verificar los programas: `gcc.exe` (C compiler), `g++.exe` (C++ compiler) y `mingw32-make.exe` (make program). Presionar **OK**.

Advertencia. Este paso requiere haber completado antes la instalación de MSYS2 y sus dependencias.

3.2.5. Enlazar las librerías

1. Clic derecho sobre el proyecto → **Build options...** (seleccionar el nombre del proyecto, no **Debug** ni **Release**).
2. Entrar a la pestaña **Linker settings**.
3. En **Link libraries**, presionar **Add** y agregar una por una, sin comillas y en este orden: `raylib`, `opengl32`, `gdi32` y `winmm`.
4. Presionar **OK**.

Figura 5
Asignación de librerías en Code::Blocks

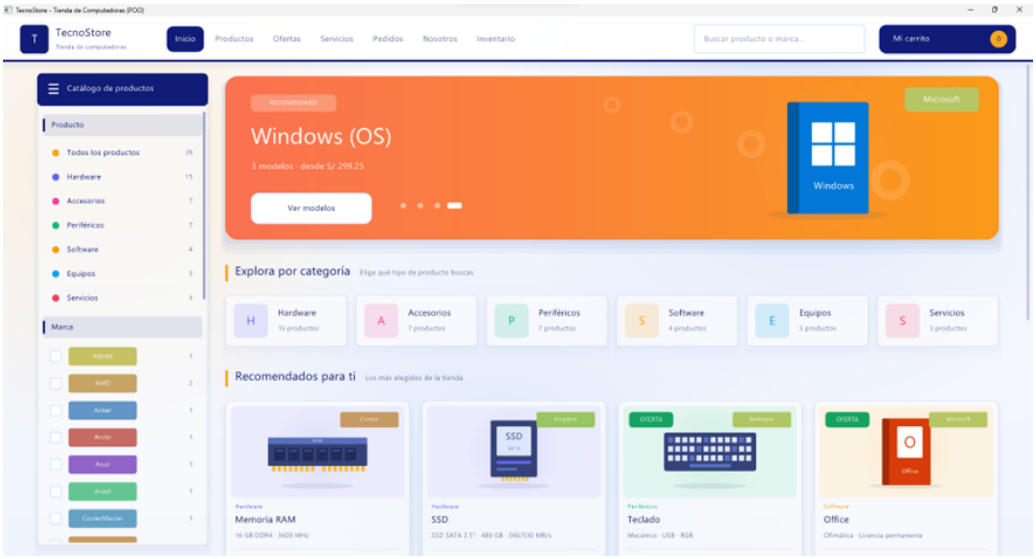


Nota. Captura de las librerías enlazadas.

3.2.6. Compilar y ejecutar

Presionar **Build and Run** (tecla **F9**). Si todo está bien configurado, se abre la ventana de TecnoStore.

Figura 6
Code::Blocks con el proyecto abierto



Nota. Captura del entorno de desarrollo con el proyecto TecnoStore.

4. Ejemplo de funcionamiento

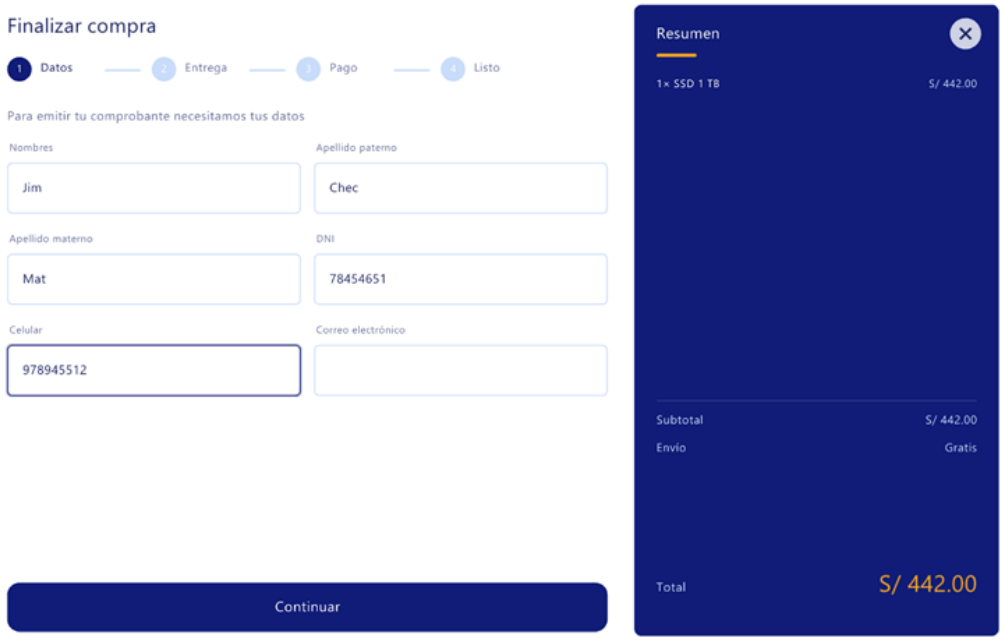
A continuación, se sigue una compra completa, con las cifras que calcula el programa.

Situación: un cliente compra un Mouse inalámbrico modelo Ergonómico y un Teclado modelo Mecánico; los recibe por delivery, usa el cupón TECNO10 y paga con tarjeta. Lo atiende el vendedor Luis (comisión 3 %).

4.1. Paso a paso en la aplicación

- En Inicio o Productos, el cliente hace clic en la tarjeta del producto. Se abre la ventana de detalle (`dibujarProducto()`).
- Elige el modelo en la lista, ajusta la cantidad y pulsa Agregar. La función `Carrito::agregar()` verifica que haya stock.
- Abre el carrito. Como lleva Mouse y Teclado, se activa el combo de S/ 10 de descuento.
- Pulsa Continuar compra. En el paso Datos escribe nombres, DNI, celular y correo; `validar()` revisa cada campo.

Figura 7
Menú de compra de un producto

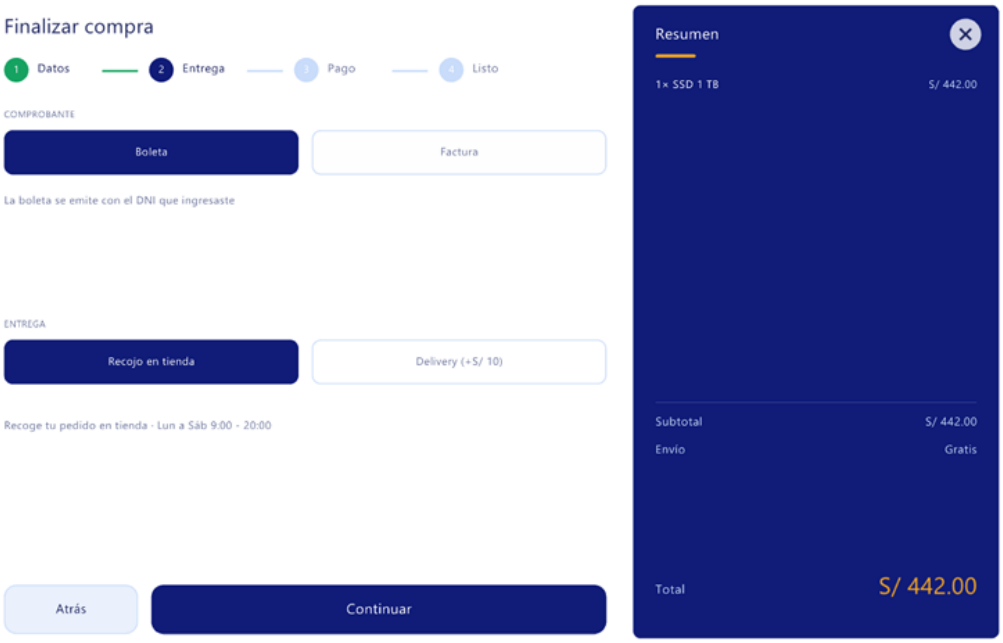


Nota. Ventana de detalle con modelos, cantidad y botón Agregar.

En Entrega elige Boleta y Delivery e ingresa su dirección.

- En Pago elige vendedor, medio de pago (Tarjeta) y escribe el cupón TECNO10; luego pulsa Aplicar.

Figura 8
Menú de finalizar compra



Nota. Paso de pago con vendedor, medio de pago y cupón.

- Pulsa Pagar. Se llama a `Tienda::pagar()`, que genera el comprobante, descuenta el stock y guarda el inventario; la pestaña Pedidos muestra la compra, con la comisión del vendedor.

4.2. Cálculo del total

Concepto	**Monto**
Mouse inalámbrico, modelo Ergonómico (S/ 50 × 1,0)	S/ 50.00
Teclado, modelo Mecánico (S/ 70 × 1,4)	S/ 98.00
Subtotal	S/ 148.00
Combo Mouse + Teclado	−S/ 10.00
Promoción por volumen (requiere 3 o más productos)	S/ 0.00
Monto neto	S/ 138.00
Cupón TECNO10 (10 % de 138,00)	−S/ 13.80
Delivery	+S/ 10.00
Base	S/ 134.20
Recargo por tarjeta (3 % de 134,20)	+S/ 4.03
TOTAL A PAGAR	**S/ 138.23**

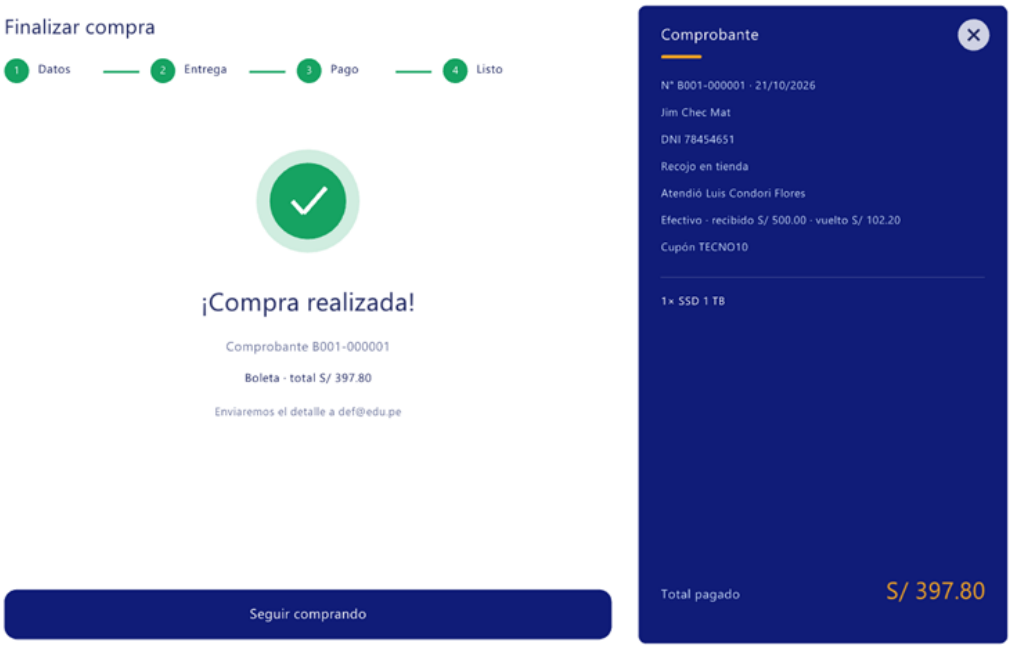
Nota. Ejemplo de operación de la compra.

Resultado en el pedido:

- Comprobante **B001-000001** (Boleta), por un total de S/ 138.23.
- Comisión del vendedor Luis: 3 % de S/ 138.23, es decir, S/ 4.15.
- Stock actualizado: el Mouse inalámbrico baja de 25 a 24 unidades y el Teclado de 18 a 17.
- El carrito se vacía, el cupón se desactiva y el inventario se guarda en `img/inventario.dat`.

Observación. La comisión se calcula sobre el total pagado, que incluye el recargo de la tarjeta y el costo de delivery.

Figura 9
Compra finalizada



Nota. Comprobante generado al finalizar la compra.

4.3. Cupones disponibles

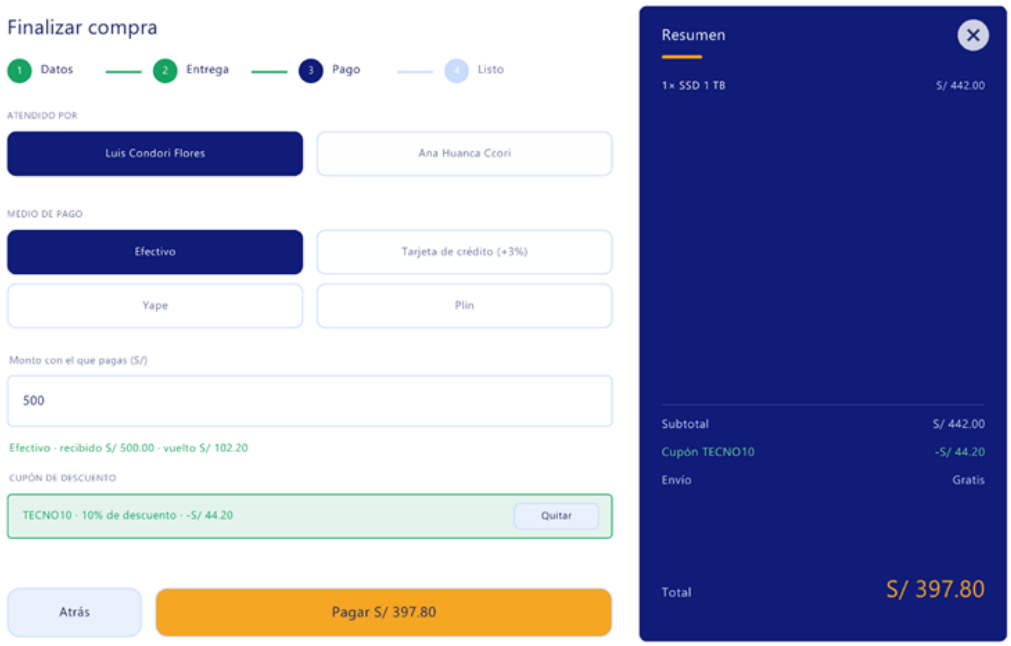
Los cupones tienen el rol de dar un descuento o sea reducir el precio total de los productos, en este caso se presentan los siguientes códigos disponibles:

Código	**Descuento**	**Condición**
TECNO10	10 %	Sin mínimo
BIENVENIDO	S/ 20	Compra mínima de S/ 100
GAMER50	S/ 50	Compra mínima de S/ 1000
PRIMERA5	5 %	Sin mínimo
ACEITUNO	90 %	Sin mínimo (*easter egg* del proyecto)

Nota. Los códigos no distinguen mayúsculas de minúsculas, porque el programa los convierte a mayúsculas antes de buscarlos. El cupón ACEITUNO es un código oculto, incluido como detalle humorístico del equipo.

El descuento del cupón se calcula sobre el monto neto (subtotal menos promociones), nunca puede superar ese monto y solo se aplica si la compra alcanza el mínimo exigido. Solo puede haber un cupón activo a la vez.

Figura 10
Cupones aplicados



Nota. Ventana de pago con un cupón activo.

5. Funciones del código

Las funciones se presentan en el mismo orden en que aparecen en el archivo. Cuando varias clases hijas hacen algo muy parecido (por ejemplo, el método `detalle()` de cada producto), se explican juntas.

5.1. Utilidades básicas

Función	Qué hace
`dec(v, n)`	Convierte un número a texto con `n` decimales sin depender de la configuración regional del sistema.
`ceros(n, ancho)`	Rellena un número con ceros a la izquierda (se usa en los números de comprobante, como B001-000001).
`limitar(v, min, max)`	Obliga a que un valor quede dentro de un rango.
`dinero(v)`	Convierte un número en texto con formato de soles, por ejemplo S/ 25.50.
`minusc(s)`	Devuelve el texto en minúsculas; sirve para buscar sin distinguir mayúsculas.

Función	Qué hace
`mayusc(s)`	Devuelve el texto en mayúsculas; se usa con los códigos de cupón.
`dividir(s, sep)`	Corta un texto en partes usando un separador (por defecto |) y devuelve una lista.

5.2. Dominio: personas

Función	Qué hace
`Persona::nombreCompleto()`	Une nombres y apellidos paterno y materno.
`Persona::datos()`	Devuelve los datos personales (DNI, teléfono, correo).
`Persona::rol()`	Método abstracto: cada clase hija dice qué papel cumple.
`Cliente::rol()`	Devuelve “Cliente persona” o “Cliente empresa”.
`Cliente::direccion()`	Entrega la dirección del cliente.
`Proveedor::rol()`	Indica si el proveedor es nacional o internacional.
`Proveedor::razon()`	Devuelve la razón social.
`Proveedor::detalle()`	Muestra el RUC y el país del proveedor.
`Trabajador::codigo()`, `sueldo()`	Devuelven el código y el sueldo del empleado.
`Trabajador::comision(monto)`	Devuelve 0 por defecto; las hijas lo redefinen.
`Cajero::rol()`	Muestra el horario de trabajo.
`Vendedor::rol()`	Muestra su porcentaje de comisión.
`Vendedor::comision(v)`	Calcula $v \times$ porcentaje$/100$.
`Tecnico::rol()`	Muestra su especialidad.
`Gerente::rol()`, `comision(v)`	Devuelve “Gerente” y calcula su porcentaje de las ventas.

5.3. Dominio: productos

5.3.1. Funciones de apoyo para los modelos

Función	Qué hace
`parseModelos(s)`	Convierte un texto como `"Básico:0.8|Estándar:1*"` en una lista de modelos (nombre, factor de precio y si es el recomendado, marcado con $\star$).
`modelosDeMarca(marca)`	Devuelve modelos típicos de una marca (Intel, AMD, Nvidia…) para el formulario de producto nuevo; si no la reconoce, da una lista genérica.
`textoModelosElegidos(v)`	Convierte los modelos marcados en el formulario al formato de texto `Nombre:1|Nombre:1`, dejando el primero como recomendado.
`modelosAmpliados()`	Tabla con listas largas de modelos para cada producto del catálogo, según su código (por ejemplo `CPU-01`).
`desdePrecio(p)`	Busca el precio más bajo entre todos los modelos de un producto (el "Desde S/…" de las tarjetas).

5.3.2. Clase Producto (base)

Función	Qué hace
`Producto(ficha, cat)`	Constructor: guarda código, nombre, marca, precio, garantía, stock y modelos; lanza error si el precio es negativo.
`codigo()`, `nombre()`, `marca()`, `categoria()`, `garantia()`, `precioBase()`, `stock()`, `modelos()`	Devuelven cada dato del producto (encapsulamiento).
`fijarModelos(s)`	Reemplaza la lista de modelos.
`modelosTexto()`	Convierte los modelos otra vez a texto para guardarlos en el archivo.
`recomendado()`	Devuelve la posición del modelo recomendado.
`precioModelo(m)`	Precio base multiplicado por el factor del modelo elegido.
`disponible()`	Indica si hay stock.
`estado()`	Devuelve "Agotado", "Pocas unidades" (5 o menos) o "Disponible".
`quitarStock(n)`	Resta unidades al vender; nunca baja de 0.
`fijarStock(n)`	Establece el stock directamente.

Función	Qué hace
`marcarOferta()`, `enOferta()`	Marca un producto como oferta y consulta si lo es.
`precioFinal(m)`	Método virtual: por defecto es `precioModelo(m)`; algunas hijas lo cambian.
`detalle()`	Método abstracto: cada producto describe sus características.

5.3.3. Clases hijas de producto

Función	Qué hace
`ProductoPersonalizado`	Producto creado desde la pestaña Inventario; su `detalle()` es el texto que escribió el usuario.
Componentes: `CPU`, `GPU`, `RAM`, `SSD`, `HDD`, `PlacaMadre`, `FuentePoder`, `Ventilador`	Cada una redefine `detalle()` para mostrar sus datos técnicos (núcleos, VRAM, GB, socket, vatios...). `Almacenamiento` es la base de SSD y HDD, que le anteponen "SSD" o "HDD".
Accesorios: `MemoriaFlash`, `PastaTermica`, `Cable`, `Adaptador`, `Cargador`	Su `detalle()` muestra tipo, capacidad, conector o compatibilidad.
`Periferico::conectividad()`	Arma el texto común de los periféricos (conector, inalámbrico, Bluetooth).
Periféricos: `Mouse`, `Teclado`, `Parlante`, `CamaraWeb`, `Microfono`, `AdaptadorInalambrico`	Cada uno agrega a `conectividad()` lo propio (DPI, RGB, vatios, resolución...).
`Software::licencia()`	Indica si la licencia es permanente o de varios meses.
`Software::precioFinal()`	Aplica un 5 % de descuento por ser licencia digital.
`Software::enOferta()`	Siempre devuelve verdadero.
`SistemaOperativo`, `Aplicacion`	Solo redefinen `detalle()`.
`EquipoCompleto::especificaciones()`	Une procesador, RAM y almacenamiento en un texto.
`EquipoCompleto::precioFinal()`	Suma S/ 10 por cada mes de garantía.
`Computadora`, `Laptop`, `Impresora`	Redefinen `detalle()` con sus características.

Función	Qué hace
`Servicio::detalle()`	Devuelve la descripción del servicio.
`Garantia::detalle()`	Agrega el tipo de diagnóstico.
`Mantenimiento::detalle()`	Agrega la vigencia en meses.

5.4. Dominio: pago, comprobante, envío, promociones y cupones

Función	Qué hace
`MedioPago::nombre()`	Texto que se muestra en el botón del medio de pago.
`MedioPago::campo()`	Indica qué dato se pide al cliente (monto, tarjeta o celular).
`MedioPago::validar(dato, total)`	Devuelve un mensaje de error, o vacío si el dato es válido.
`MedioPago::resumen(dato, total)`	Texto que queda registrado en el pedido.
`MedioPago::recargo(monto)`	Recargo adicional; vale 0 por defecto.
`Efectivo`	Valida que el monto cubra el total y muestra el vuelto en el resumen.
`Tarjeta`	Pide los 4 últimos dígitos; `recargo()` suma 3 %.
`Billetera`, `Yape`, `Plin`	Validan un celular de 9 dígitos que empiece con 9; Yape y Plin solo le ponen nombre.
`Comprobante::nombre()`, `serie()`, `pideRUC()`, `emitir(total)`	Interfaz común de los comprobantes.
`Boleta`	Serie B001; no pide RUC.
`Factura`	Serie F001; pide RUC y muestra el IGV (18 %).
`Envio::nombre()`, `costo()`, `pideDireccion()`	Interfaz común de los envíos.
`RecojoEnTienda`	Costo 0; no pide dirección.
`Delivery`	Costo S/ 10; pide dirección.
`cantidadPorNombre(l, nombre)`	Cuenta cuántas unidades de un producto hay en el carrito.
`subtotalDe(l)`	Suma precio × cantidad de todas las líneas.
`Promocion::nombre()`, `descuento(l)`	Interfaz común de las promociones.

Función	Qué hace
`PromoCombo`	Descuenta un monto fijo por cada pareja de productos (Mouse + Teclado).
`PromoVolumen`	Si hay 3 o más productos, descuenta 5 % del subtotal.
`Cupon::codigo()`, `descripcion()`, `minimo()`	Datos del cupón.
`Cupon::calcular(neto)`	Método abstracto: cada tipo calcula su descuento.
`Cupon::descuento(neto)`	Devuelve 0 si no se llega al mínimo; si sí, el descuento sin superar el total.
`CuponPorcentaje`, `CuponMonto`	Descuentan un porcentaje o un monto fijo.

5.4.1. Clase Carrito

Función	Qué hace
`agregarPromo(p)`	Registra una promoción en el carrito.
`agregar(p, modelo, n)`	Agrega unidades respetando el stock y devuelve cuántas agregó; si la línea ya existe, suma la cantidad.
`quitar(i)`	Resta una unidad de la línea y la elimina si llega a 0.
`vaciar()`	Borra todo el carrito.
`lineas()`	Devuelve las líneas actuales.
`cantidad()`	Total de unidades en el carrito.
`cantidadDe(p)`	Unidades de un producto concreto.
`subtotal()`	Suma sin descuentos.
`detalleDescuentos()`	Lista de promociones que sí se aplican y cuánto descuenta cada una.
`descuentos()`	Suma de todos esos descuentos.

5.5. Dominio: clase Tienda

Función	Qué hace
`agregarAlCatalogo<T>(...)`	Plantilla (privada): crea un producto de la clase `T` y lo agrega al catálogo.

Función	Qué hace
`nombresDe(lista)`	Devuelve los nombres de una lista de objetos (medios de pago, envíos…).
`limpio(s)`	Quita caracteres que romperían el archivo (`|`, saltos de línea, tabulaciones).
`guardarInventario()`	Escribe en `img/inventario.dat` el stock de cada producto y los datos completos de los personalizados.
`cargarInventario()`	Lee ese archivo al iniciar: restaura el stock y recrea los productos personalizados; ignora filas dañadas.
`crearCatalogoBase()`, `crearOpcionesDeCompra()`, `crearPersonal()`, `crearPromocionesYCupones()`	Funciones privadas que cargan los datos precargados de la tienda.
`Tienda()`	Constructor: crea el catálogo, medios de pago, comprobantes, envíos, trabajadores, proveedores, promociones y cupones; marca ofertas, amplía modelos y carga el inventario guardado.
`productos()`	Devuelve la lista de todos los productos.
`siguienteCodigo()`	Busca el primer código libre (PRD-001, PRD-002…).
`agregarPersonalizado(...)`	Valida y crea un producto nuevo y lo guarda en el archivo.
`reponerStock(codigo, cantidad)`	Suma stock a un producto y guarda.
`marcas()`	Lista de marcas sin repetir.
`vendedores()`	Usa `dynamic_cast` para filtrar solo los trabajadores que son Vendedor.
`equipo()`, `proveedores()`, `pedidos()`, `carrito()`	Dan acceso a esos datos.
`nombresMedios()`, `nombresComprobantes()`, `nombresEnvios()`	Nombres para los botones de elección.
`medio()`, `comprobante()`, `envio()`	Posición elegida actualmente.
`elegirMedio()`, `elegirComprobante()`, `elegirEnvio()`	Cambian la elección.
`medioActual()`, `comprobanteActual()`, `envioActual()`	Devuelven el objeto elegido.

Función	Qué hace
`aplicarCupon(texto)`	Busca el cupón (sin distinguir mayúsculas), revisa la compra mínima y lo activa; devuelve vacío si funcionó o el motivo del rechazo.
`quitarCupon()`, `cupon()`	Quita o consulta el cupón activo.
`descuentoCupon()`	Cuánto descuenta el cupón sobre el subtotal ya con promociones.
`base()`	Subtotal − descuentos − cupón + costo de envío.
`recargo()`	Recargo del medio de pago sobre esa base.
`total()`	Base + recargo; es lo que paga el cliente.
`pagar(...)`	Cierra la venta: arma número y fecha del comprobante, datos del cliente y entrega, calcula la comisión, descuenta el stock, guarda el inventario, vacía el carrito y registra el pedido.

5.6. Interfaz: funciones de dibujo

Función	Qué hace
`R(x, y, w, h)`	Atajo para crear un rectángulo.
`cargarFuente()`	Busca una fuente del sistema (Segoe UI, Arial, DejaVu…) y la carga; si no encuentra, usa la de raylib.
`textura(ruta)`	Carga una imagen una sola vez y la guarda; si no existe, devuelve nulo.
`Mezcla(a, b, t)`	Mezcla dos colores según el valor `t`.
`Al(color, alfa)`	Cambia la transparencia de un color.
`imagen(ruta, area)`	Dibuja una imagen centrada y sin deformarla; devuelve falso si no existe.
`Texto`, `Ancho`, `TextoCentro`, `TextoDer`	Dibujan texto, miden su ancho, lo centran en un rectángulo o lo alinean a la derecha.
`Ajustar(s, tam, max)`	Acorta un texto con `..` si no cabe en el ancho.
`LineasR`, `CirculoDeg`	Envoltorios para que el programa funcione con distintas versiones de raylib.
`Caja(...)`	Rectángulo redondeado con borde.
`Sombra(...)`	Sombra suave detrás de un panel.

Función	Qué hace
`Vidrio(...)`	Panel blanco semitransparente con sombra y brillo (el estilo de la aplicación).
`Deg(...)`	Rectángulo con degradado y esquinas redondeadas.
`colorMarca(marca)`	Genera un color fijo a partir del nombre de la marca.
`Marca(marca, area)`	Dibuja el logo de la marca (`img/marcas/`) o, si no hay, una etiqueta de color con el nombre.
`contienePalabra(texto, palabra)`	Revisa si la palabra aparece como palabra completa (para no confundir "ram" con "programa").
`clasificar(nombre)`	Recorre la tabla `REGLAS` y decide qué tipo de dibujo corresponde (gpu, cpu, mouse…).
`arte(producto, area, color)`	Dibuja la foto del producto o, si no hay, una ilustración hecha con formas geométricas: usa `clasificar()` para elegir la función `dibujarGPU()`, `dibujarCPU()`, `dibujarMouse()`, etc.
`Lienzo`	Estructura con las herramientas de dibujo que usan esas ilustraciones: `rect` (rectángulo), `circulo`, `aro` (contorno), `linea`, `texto`, `aspas` (aspas que giran) y `cajaSoftware`.

5.7. Interfaz: elementos reutilizables

Función	Qué hace
`ElementoUI::mover(r)`	Fija la posición del elemento.
`ElementoUI::sobre(mouse)`	Indica si el mouse está encima.
`ElementoUI::actualizar(dt, mouse)`	Anima suavemente el efecto al pasar el mouse.
`ElementoUI::dibujar(foco)`	Método abstracto que cada elemento implementa.
`Campo::teclear(ch)`	Agrega una letra según el modo (texto libre, solo dígitos o decimal); respeta el largo máximo y admite tildes y ñ.
`Campo::borrar()`	Elimina el último carácter sin romper letras con tilde.
`Campo::dibujar(foco)`	Dibuja etiqueta, borde (rojo si hay error), texto, cursor parpadeante y mensaje de error.
`Pantalla::dibujar(ox, cw, y)`	Método abstracto: cada pantalla se dibuja a sí misma y devuelve la altura que ocupó.

Función	Qué hace
`PantallaNosotros`	Muestra la información de la tienda, el equipo de trabajo y los proveedores.
`PantallaPedidos`	Historial de compras, del más reciente al más antiguo, con comisión y cupón usado.

5.8. Clase Interfaz

5.8.1. Utilidades

Función	Qué hace
`anim(id, activo)`	Devuelve un valor de 0 a 1 que sube o baja suavemente (efectos al pasar el mouse).
`sobre(r)`	Indica si el mouse está sobre un rectángulo en la capa actual.
`luego(f)`	Guarda una acción para ejecutarla al final del cuadro.
`ir(capa)`	Pide cambiar de capa: 0 página, 1 carrito, 2 pago, 3 detalle.
`aviso(texto)`	Muestra un mensaje temporal arriba.
`abrir(producto)`	Abre el detalle de un producto con su modelo recomendado.
`cambiarHero(i)`	Cambia el producto destacado del banner de Inicio.
`filtro(...)`	Aplica pestaña, categoría, producto y marca, y regresa al inicio de la lista.
`columnas(ancho)`	Calcula cuántas tarjetas caben por fila.
`velo(a, k)`	Oscurece el fondo detrás de las ventanas emergentes.
`boton(...)`	Dibuja un botón y devuelve verdadero si se le hizo clic.
`chip(...)`	Botón de opción; el elegido se pinta de azul.
`equis(...)`	Botón redondo de cerrar.
`campo(c, r)`	Coloca, actualiza y dibuja un `Campo`; le da el foco si se hace clic.
`tag(...)`	Etiqueta pequeña de color (por ejemplo, “Disponible · 12”).
`casilla(...)`	Casilla con visto.
`estadoColor(p)`	Rojo si está agotado, naranja si hay pocas unidades, verde si hay stock.
`filtrar()`	Devuelve los productos que cumplen pestaña, categoría, marca y búsqueda, ordenados según el criterio elegido.

Función	Qué hace
`tarjeta(p, r, id)`	Dibuja la tarjeta de un producto y la abre al hacer clic.
`titulo(...), seccion(...)`	Encabezados de sección.
`barraLateral(x, w)`	Panel con categorías, productos de la categoría abierta y marcas, con contadores y desplazamiento.

5.8.2. Páginas y ventanas

Función	Qué hace
`pie(...)`	Pie de página con horario y contacto; devuelve la altura usada.
`inicio(...)`	Pantalla principal: banner rotativo, categorías, recomendados, marcas y beneficios.
`catalogo(...)`	Sirve para Productos, Ofertas y Servicios: título, ordenamiento, filtro de marca y cuadrícula de tarjetas.
`inventario(...)`	Formulario de producto nuevo y lista de stock con botones +1 y Agregar N.
`dibujarPagina()`	Elige qué página dibujar según la pestaña y calcula la barra de desplazamiento.
`dibujarTopbar()`	Barra superior: logo, pestañas, buscador y botón del carrito con contador.
`dibujarCarrito()`	Panel lateral con líneas del carrito, botones +/−, descuentos y total parcial.
`camposPaso()`	Lista de casillas del paso actual (se usa con la tecla Tab).
`validar()`	Revisa los datos del paso: nombres, DNI de 8 dígitos, celular, correo, RUC, dirección o dato del medio de pago.
`avanzar()`	Pasa al siguiente paso; en el último crea el `Cliente` y llama a `Tienda::pagar()`.
`dibujarCheckout()`	Ventana de compra con 4 pasos (Datos, Entrega, Pago, Listo) y panel de resumen o comprobante.
`dibujarProducto()`	Ventana de detalle: ilustración, lista de modelos con buscador, cantidad y botón Agregar.
`dibujarAviso()`	Dibuja el mensaje temporal.
`fondo()`	Dibuja el fondo claro con manchas de color animadas.

5.8.3. Ciclo principal

Función	Qué hace
`paso()`	Se ejecuta 60 veces por segundo y llama, en orden, a las tres funciones siguientes.
`leerEntradaYActualizar()`	Lee mouse y teclado, escala la interfaz a la ventana, anima paneles y procesa el texto escrito.
`dibujarTodo()`	Dibuja todas las capas: fondo, página, barra superior, carrito, pago, detalle de producto y aviso.
`cerrarCuadro()`	Cambia de capa, ejecuta las acciones pendientes guardadas con `luego()` y elige el cursor del mouse.
`Interfaz()`	Constructor: prepara las marcas y elige los productos destacados y recomendados.
`ejecutar()`	Crea la ventana, la ajusta a la pantalla, carga la fuente, corre el ciclo hasta cerrar y libera las imágenes.
`main()`	Crea la `Interfaz` y la ejecuta.

6. Limitaciones y mejoras futuras

Pese a todo lo expuesto, aún existen aspectos que se pueden mejorar o ampliar para lograr un proyecto más completo y consistente. Se agrupan en tres áreas.

6.1. Funcionalidad

- **Guardar los pedidos:** actualmente los pedidos y la numeración de los comprobantes se pierden al cerrar el programa, por lo que al reiniciar la serie vuelve a B001-000001.
- **Exportar comprobantes:** generar boletas y facturas en un archivo (por ejemplo, PDF) cuando el cliente lo solicite.
- **Roles de administrador y usuario:** limitar funciones como el inventario para evitar modificaciones no solicitadas a la tienda.
- **Revisar el stock al pagar:** volver a comprobar las existencias en el momento de cerrar la venta, por si cambiaron mientras el cliente compraba.
- **Base de la comisión:** calcularla solo sobre el monto de los productos, sin incluir el recargo de la tarjeta ni el delivery.
- **Validaciones más estrictas:** por ejemplo, verificar el dígito de control del RUC y limitar el campo de monto en efectivo a un único punto decimal.

6.2. Estructura del código

- **Dividir el archivo:** separar `main.cpp` en archivos de cabecera y de implementación (personas, productos, ventas, tienda e interfaz) para compilar y trabajar en equipo con mayor comodidad.
- **Dividir la clase Interfaz:** crear una clase por pantalla, siguiendo el patrón de `PantallaNosotros` y `PantallaPedidos`.
- **Datos externos:** mover el catálogo y las tablas de modelos a un archivo de datos, en lugar de escribirlos dentro del código.
- **Includes específicos:** reemplazar `<bits/stdc++.h>`, que solo existe en GCC, por las cabeceras estándar que realmente se usan, para que el código compile en otros compiladores.

6.3. Interfaz

- Remodelar y organizar de mejor manera la interfaz general.
- Avisar claramente al cliente cuando un cupón ya aplicado deja de cumplir su compra mínima tras quitar productos del carrito.

CONCLUSIONES

PRIMERA: Se logró desarrollar una tienda de computadoras en C++ que aplica los cuatro pilares de la POO en un caso realista: abstracción (clases base), herencia (jerarquías de productos, personas y pagos), polimorfismo (precios, recargos, comisiones y validaciones) y encapsulamiento (atributos protegidos con sus getters correspondientes).

SEGUNDA: La separación entre dominio e interfaz permite modificar la forma en que se estructura el negocio sin alterar la presentación de la tienda, y viceversa. Además, la lógica puede probarse de forma independiente de la pantalla.

TERCERA: El diseño del código es extensible: gracias a la abstracción y al uso de punteros a la clase base, nuevos medios de pago, cupones o productos se incorporan creando clases nuevas, sin modificar el resto del programa.

CUARTA: El inventario se guarda en un archivo y se recupera al iniciar la aplicación. Sin embargo, aún no se conservan los pedidos ni la numeración de los comprobantes, lo que queda como mejora futura.

QUINTA: El desarrollo del proyecto permitió comprender mejor cómo se combinan la herencia y la composición según el caso: los productos se organizan en jerarquías de clases, mientras que la tienda reúne todos los elementos mediante composición.

ANEXOS

Anexo 1

Enlace del video de demostración y explicación del proyecto

`https://drive.google.com/file/d/168Y22QBmH3TcarbfVL0Tpzig_2Y6_P_i/view?usp=sharing`

Anexo 2

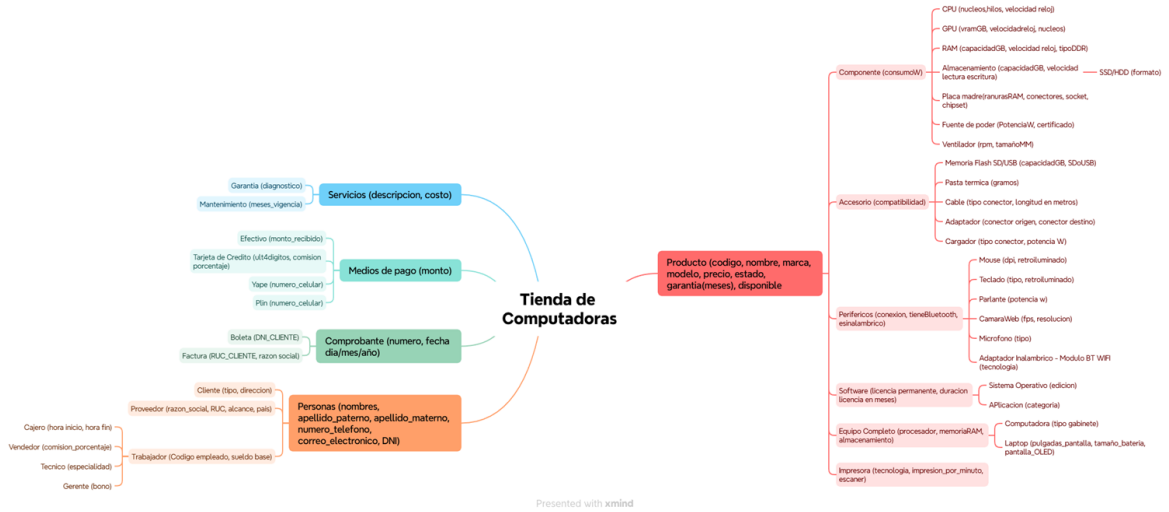
Enlace del código fuente completo (Compilado en C++)

`https://github.com/cristianchecalla-byte-unap/POO_TecnoStore.git`

Nota. Se pueden adjuntar fotos en la subcarpeta `img/productos` (formato .png, con el código del producto como nombre) y logos en `img/marcas` (`marca.png`). Las instrucciones de compilación se encuentran en la sección de instalación y ejecución.

Anexo 3

Diagrama principal usado



Nota. Elaboración propia (creado desde XMind).